



# ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

## LABORATORY REPORT

### Microprocessor and Microcontrollers Lab

Regulations : AR23

Course Code : 231EC6L02

Semester : VI

#### Submitted by

Student Name :

Roll Number :

Section :



## **List of Experiments**

### **PART- A: 8086 Assembly Language Programming and Interfacing**

1. Programs for 16 -bit arithmetic operations (using Various Addressing Modes).
  - a. Addition and subtraction of n-BCD numbers.
  - b. Multiplication and Division operations.
  - c. Addition of an array of numbers with overflow detection.
2. Program for sorting an array.
3. Program for Factorial of given n-number.
4. Interface DAC to 8086.
5. Interface stepper motor to 8086.

### **PART-B: 8051 Assembly Language Programming and Interfacing**

6. Finding number of 1's and number of 0's in a given 8-bit number.
7. Average of n-numbers.
8. Timer control based square wave generation program using 8051.
9. Interface of UART with 8051.
10. Interface LCD to 8051.

### **List of Augmented Experiments:**

11. Interface of 8086 with Traffic Light Controller.
12. Interface keyboard to 8051.



## INTRODUCTION TO MASM/TASM

### ASSEMBLY LANGUAGE PROGRAMMING USING MASM SOFTWARE:

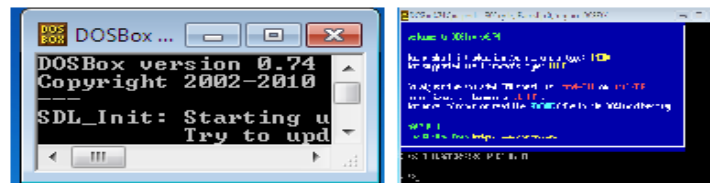
The programs are written using assembly language in editor then compile it. The compiler converts assembly language statements into machine language statements/checks for errors. Then execute the compiled program.

There are different soft wares developed by different companies for assembly language programming, they are:

- **MASM** - Microsoft Company
- **TASM** - Bore Land Company

### HOW TO ENTER INTO MASM EDITOR:

→ Click **DOS BOX** icon on the desktop.



→ Path setting

Suppose it display path as **Z:\>**

1. Type **MOUNT C C:\8086** then drive c is mounted as local directory C:\MASM\ then the path is **Z :\>**
2. Type **C:**
3. Type edit i.e.; **C:\>edit** then enter into **MASM** text editor
4. Go to **FILE** and select **NEW** and write the **ALP** (Assembly Language Program) in this editor.
5. Save the file as **Filename.ASM** then exit from the editor and go to prompt.
6. To check for errors, type **C:\> MASM filename.ASM**
7. To link this file, type **C: \>LINK filename.OBJ** i.e., link the program in assembly with **DOS**
8. To Debug the executable file, type **C :\> DEBUG filename. EXE** .Then it display “-” on the screen
9. Type ‘**R**’ to display the registers contents and starting step of the program.
10. Type ‘**T**’ to enter into step by step execution of program.

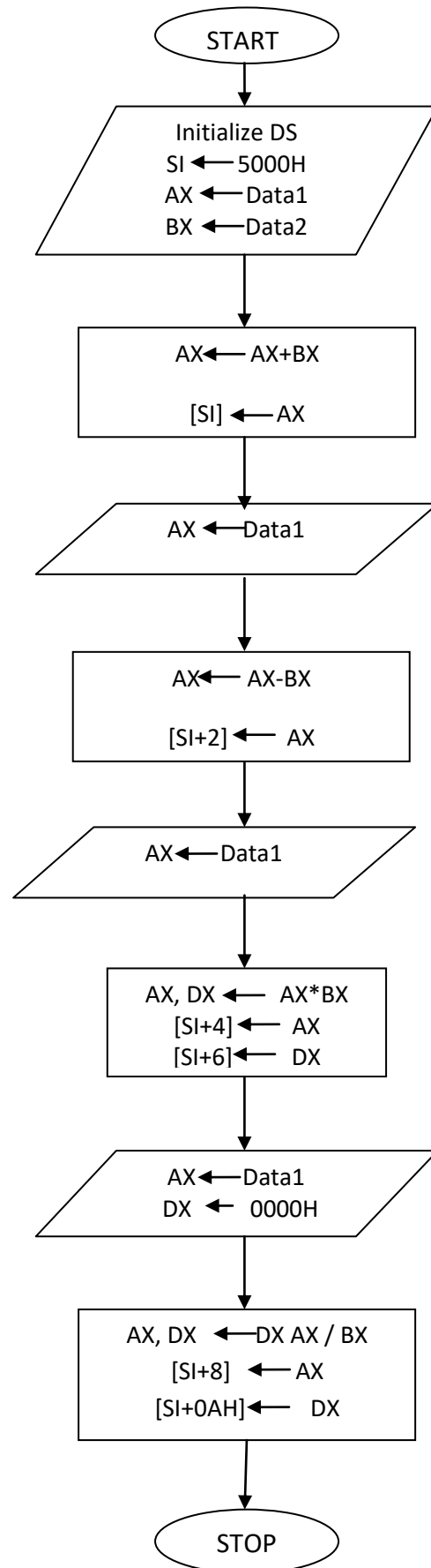
### DEBUG COMMANDS:

|            |           |                                    |
|------------|-----------|------------------------------------|
| DISPLAY    | D [range] | ; To Display contents of memory    |
| UNASSEMBLE | U [range] | ; To Display Code Table (Hex file) |
| QUIT       | Q         | ; To exit from debugging           |



# **ARITHMETIC OPERATIONS ON 16-BIT DATA**

## FLOW CHART:





**Exp No: 1****Date:****ARITHMETIC OPERATIONS ON 16-BIT DATA****AIM:** Assembly language program to perform all arithmetic operations on 16bit data**PORTS USED:** None**REGISTERS USED:** AX, BX, SI**ALGORITHM:****Step1:** Start**Step2:** Initialize data segment**Step3:** Initialize SI with some memory location**Step4:** Load the given data to registers AX & BX**Step5:** Perform addition and Store the result**Step6:** Repeat step 4**Step7:** Perform subtraction and Store the result**Step8:** Repeat step 4**Step9:** Perform multiplication and Store the result**Step10:** Repeat step 4**Step11:** Perform division and Store the result**Step12:** Stop

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**PROGRAM:**

ASSUME CS: CODE, DS: DATA

DATA SEGMENT

N1 EQU 8888H

N2 EQU 4444H

DATA ENDS

CODE SEGMENT

START: MOV AX, DATA

MOV DS, AX

MOV SI, 5000H

MOV AX, N1

MOV BX, N2

ADD AX, BX

MOV [SI], AX

MOV AX, N1

SUB AX, BX

MOV [SI+2], AX

MOV AX, N1

MUL BX

MOV [SI+4], AX

MOV [SI+6], DX

MOV AX, N1

MOV DX, 0000

DIV BX

MOV [SI+8], AX

MOV [SI+0AH], DX

MOV AH, 4CH

INT 21H

CODE ENDS

END START

**MANUAL CALCULATIONS:**

**RESULT:**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**Exp No: 1****Date:****ADDITION AND SUBTRACTION OF BCD NUMBERS:****PROGRAM:**

ASSUME CS:CODE,DS:DATA

DATA SEGMENT

NUM1 DB 56H

NUM2 DB 24H

SUM DB ?

DIFF DB ?

DATA ENDS

CODE SEGMENT

START: MOV AX,DATA

MOV DS,AX

MOV AL,NUM1

ADD AL,NUM2

DAA

MOV SUM,AL

MOV AL, NUM1

SUB AL, NUM2

DAS

MOV DIFF,AL

MOV AH,4CH

INT 21H

CODE ENDS

END START

**MANUAL CALCULATIONS:**



**RESULT:**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**Exp No: 1****Date:****ADDITION OF AN ARRAY OF NUMBERS WITH OVERFLOW  
DETECTION****PROGRAM:**

ASSUME CS: CODE, DS: DATA

DATA SEGMENT

ARR DB 10H, 20H, 30H, 40H, 50H

N DB 05

SUM DW ?

OF DB ?

DATA ENDS

CODE SEGMENT

START: MOV AX,DATA

MOV DS,AX

XOR AX,AX

XOR CX,CX

MOV CL,N

MOV SI,OFFSET ARR

MOV OF,0

ADD\_LOOP: MOV BL,[SI]

ADD AX, BX

JO OVERFLOW

JC OVERFLOW

INC SI

LOOP ADD\_LOOP

MOV SUM,AX

JMP DONE

OVERFLOW: MOV OF,1

MOV SUM,AX

DONE: MOV AH,4CH

INT 21H

CODE ENDS

END START

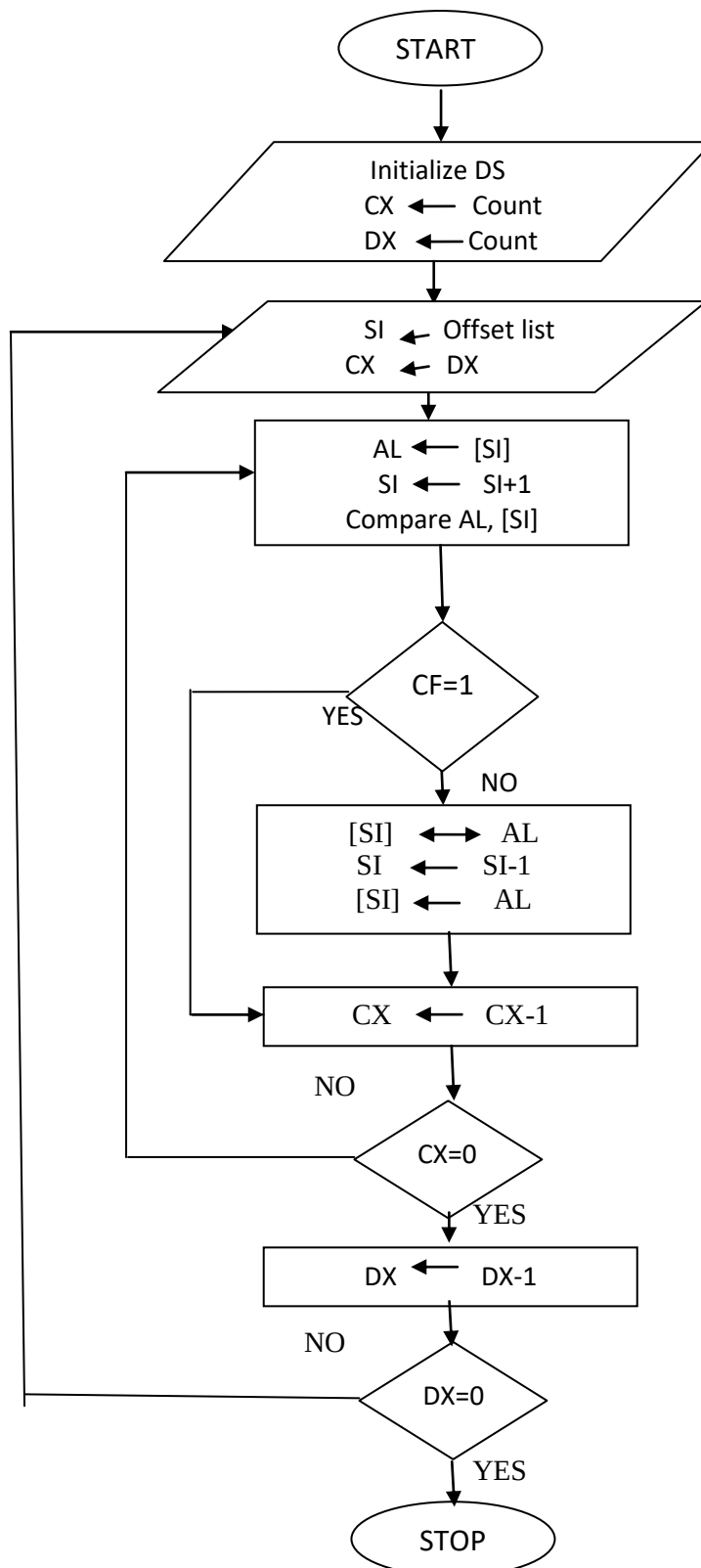
**MANUAL CALCULATIONS:**

**RESULT:**



# **SORTING AN ARRAY**

## FLOW CHART





**Exp No:2****Date:****SORTING AN ARRAY****AIM:** Assembly language program to do sorting of numbers in a given series**PORT USED:** None**REGISTERS USED:** CX, DX, AL, SI**ALGORITHM:****Step 1:** Start**Step 2:** Initialize data segment**Step 3:** Load CX register with count**Step 4:** Copy the contents from CX to DX**Step 5:** Load SI with offset list**Step 6:** Copy the contents from DX to CX**Step 7:** Move the contents from memory location SI to AL**Step 8:** Increment SI**Step 9:** Compare AL contents with [SI]**Step 10:** Jump to step15 if carry**Step 11:** Exchange the contents of AL with [SI]**Step 12:** Decrement SI**Step 13:** Move the contents from AL to memory location SI**Step 14:** Increment SI**Step 15:** Decrement CX and jump to step7 if no zero**Step 16:** Decrement DX and jump to step5 if no zero**Step 17:** Stop

**CODE TABLE:**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**PROGRAM:**

```
ASSUME CS: CODE, DS: DATA
DATA SEGMENT
LIST DB 56H, 12H, 72H, 32H
COUNT EQU 0003H
DATA ENDS
CODE SEGMENT
ORG 1000H
START:
    MOV     AX, DATA
    MOV     DS, AX
    MOV     CX, COUNT
    MOV     DX, CX
AGAIN:  MOV     SI, OFFSET LIST
    MOV     CX, DX
BACK:   MOV     AL, [SI]
    INC     SI
    CMP     AL, [SI]
    JC      NEXT
    XCHG    [SI], AL
    DEC     SI
    MOV     [SI], AL
    INC     SI
NEXT:   LOOP    BACK
    DEC     DX
    JNZ     AGAIN
    MOV     AH, 4CH
    INT     21H
CODE ENDS
END START
```

**MANUAL CALCULATIONS:**

**RESULT:**



# **FACTORIAL OF GIVEN N-NUMBER**





**Exp No: 3****Date:****FACTORIAL OF GIVEN N-NUMBER**

**AIM:** Assembly language program to find the Factorial of given n-number.

**PORT USED:** None

**REGISTERS USED:**AX, CL, SI

**ALGORITHM:**

**Step1:** Start

**Step2:** Initialize data segment

**Step3:** Load CL register with count

**Step4:** Source Index register is loaded with 5000H

**Step5:**CL loaded with 8-bit count

**Step6:**AL is initialized to 01H

**Step7:** AH is initialized to 00H

**Step8:** Multiply CL with AL

**Step9:** Decrement CL, if CL not equal to Zero go to step8

**Step10:** Store the contents of AL to memory location [SI]

**Step11:** Store the contents of AH to memory location [SI+0001]

**Step12:** Stop

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**PROGRAM:**

ASSUME CS: CODE, DS: DATA

DATA SEGMENT

N1 EQU 07H

DATA ENDS

CODE SEGMENT

ORG 1000H

START: MOV AX, DATA

MOV DS, AX

MOV SI, 5000H

MOV CX, N1

MOV AL, 01H

MOV AH, 00H

BACK: MUL CX

CMP CX, 01H

JE NEXT

LOOP BACK

NEXT: MOV [SI], AX

MOV AH, 4CH

INT 21H

CODE ENDS

END START

**MANUAL CALCULATIONS:**

**RESULT:**



# **INTERFACE DAC TO 8086**





**Exp No:4****Date:****INTERFACE DAC TO 8086****AIM:** Assembly language program to generate the following wave forms:

- Triangular wave forms
- Saw tooth wave forms
- Square wave

**REGISTERS USED:** general purpose registers: AL, DX, and CX**PORTS USED:** Out (port-B)**CONNECTION:** J<sub>4</sub> of ESA 86/88 to J<sub>1</sub> DAC interface.

**DESCRIPTIONS:** As can be from the circuit only 17 lines from the connector are used totally. The port A and port B of 8255 programmable peripheral interface are used as output ports. The digital inputs to the DAC's are provided through the port A and port B of 8255. the analog outputs of the DAC's are connected to the inverting inputs of op-amps  $\mu A741$  which acts as current to voltage converters. The out puts from the op- amps are connected to points marked  $X_{out}$  and  $Y_{out}$  at which the wave forms are observed on a CRO. (port A is used to control  $X_{out}$  port B is used to control  $Y_{out}$ ). the difference voltage for the DAC's is derived from an on-board voltage regulator  $\mu A 723$  .it generates a voltage of about 8V. the offset balancing of the op-amps is done by making use of the two 10k pots provided. The output wave forms are observed at  $X_{out}$  and  $Y_{out}$  on an oscillator.

**THEORY:****BASIC DAC TECHNIQUE:**

$$V_o = K V_{FS} (d_1 \cdot 2^{-1} + d_2 \cdot 2^{-2} + \dots + d_n \cdot 2^{-n})$$

Where  $d_1$  = MSB,  $d_2$  = LSB $V_{FS}$  = Full scale reading / out put voltage

K --- Conversion factor is adjusted to 'unity'.

D/A converters consist of 'n' bit binary word 'D' and is combined with a reference voltage  $V_R$

to give an analog output. The out put can be either voltage or

current Out put voltage  $V_o = K V_{FS} (d_1 \cdot 2^{-1} + d_2 \cdot 2^{-2} + \dots + d_n \cdot 2^{-n})$

**CODE TABLE: (TRIANGULAR WAVE)**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

MSB weight =  $\frac{1}{2} V_{FS}$  if  $d_1 = 1$  and all are zero's,  $K = 1$ . LSB weight =  $V_{FS}/2^n$  if  $d_n = 1$  and all are zero's,  $K = 1$

### DUAL DAC INTERFACE:

- This program generates a square wave or a Triangular wave at points  $X_{out}$  or  $Y_{out}$  of interface. The waveforms may be observed on an oscilloscope.
- This program can be executed in STAND-ALONE MODE or SERIAL MODE of operation.
- The program starts at memory location 3000H

### ALGORITHM: (FOR TRIANGULAR WAVE):

**Step 1:** Start

**Step 2:** Initialize the control word register with all ports as simple I/O mode

**Step 3:** Load Maximum Amplitude value to CX register.

**Step 4:** Load 00 to AL register.

**Step 5:** Initialize the port A address.

**Step 6 :** Locate the contents of AL to DX register.

**Step 7:** Increment the value in AL by one.

**Step 8:** Locate AL contents to DX register.

**Step 9:** Decrement the value of CX register by one and go to step 6 if CX not equal to zero.

**Step 10:** Load 00FF to CX register.

**Step 11:** Decrement the value of AL by one.

**Step 12:** Locate the contents in AL register to DX register.

**Step 13:** Decrement the value of CX by one and go to step 12 if CX not equal to zero.

**Step 14:** Otherwise move to step 3

**Step 15:** Stop.

### TRIANGULAR WAVE

```
MOV DX, 0FFE6
```

```
MOV AL, 80
```

```
OUT DX, AL
```

```
MOV DX, 0FFE0
```

```
MOV AL, 00
```

```
RPT: MOV CX, 00FF
```

```
L1: OUT DX, AL
```

```
    INC AL
```

```
    LOOP L1
```

```
    MOV CX, 00FF
```

```
L2: OUT DX, AL
```

```
    DEC AL
```

```
    LOOP L2
```

```
    JMP RPT
```

**CODE TABLE: (SAWTOOTH WAVE)**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**SAWTOOTH WAVE**

```
MOV DX, 0FFE6
MOV AL, 80
OUT DX, AL
MOV DX, 0FFE0
MOV AX, 00
RPT: MOV CX, 0FF
L1: OUT DX, AX
    INC AX
    LOOP L1
    MOV AX, 00
    OUT DX, AX
JMP RPT
```

**SQUARE WAVE:**

```
MOV DX, 0FFE6
MOV AL, 80
OUT DX, AL
MOV DX, 0FFE0
RPT: MOV AL, 00FF
    OUT DX, AL
    CALL DELAY
    MOV AL, 00
    OUT DX, AL
    CALL DELAY
JMP RPT
```

**DELAY PROGRAM:**

```
MOV CX, 1E
L1: NOP
NOP
LOOP L1
RET
```

**CODE TABLE: (SQUARE WAVE)**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**RESULT:**





# **INTERFACE STEPPER MOTOR TO 8086**



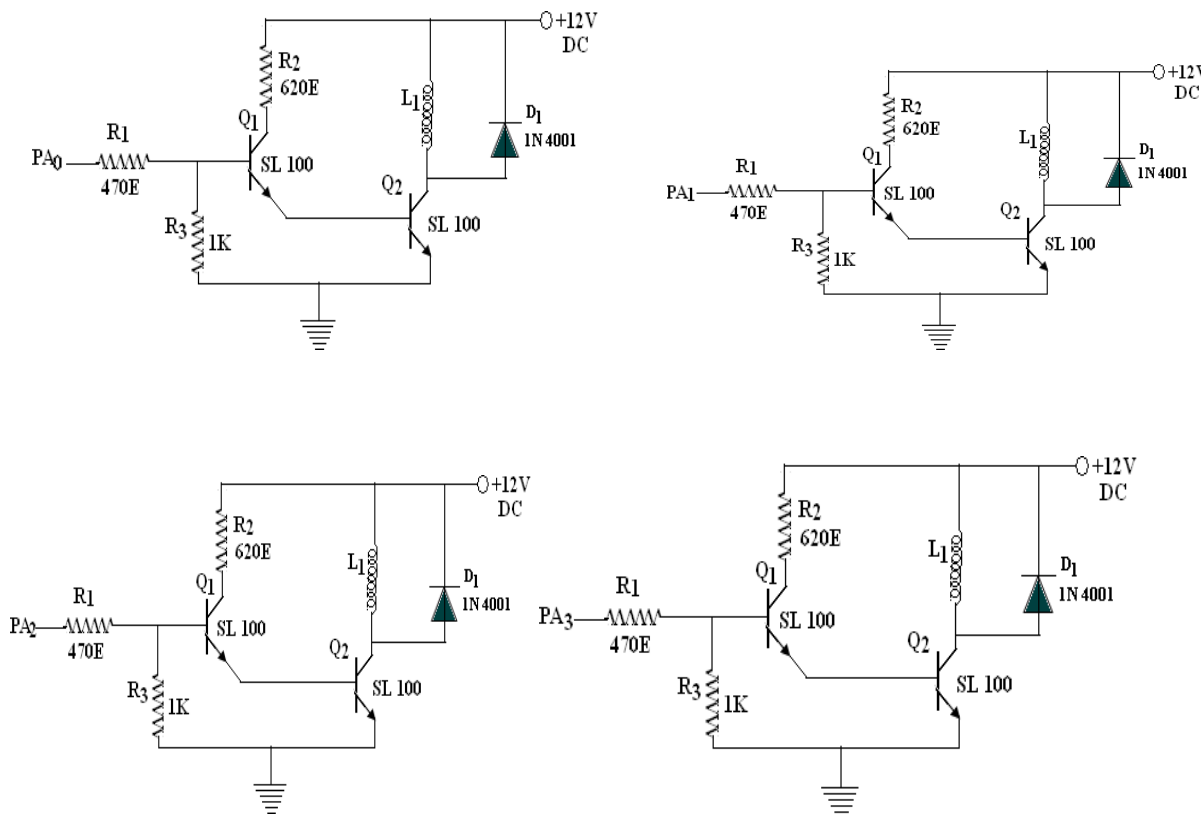
**Exp No: 5****INTERFACE STEPPER MOTOR TO 8086**

**AIM:** Assembly language program to rotate shaft of a 4 phase stepper motor in clockwise 15 rotations.

**REGISTERS USED:** General purpose registers: AL, DX, CX

**PORTS USED:** Port B, port C (out)

**CONNECTIONS:** J4 of ESA 86/88E to J<sub>1</sub> of stepper motor.

**OPERATING PRINCIPLE OF PERMANENT MAGNET STEPPER MOTOR:**

It consists of two stator windings A,B and a motor having two magnetic poles N and S. when a voltage +v is applied to stator winding A, a magnetic field  $F_a$  is generated. The rotor positions itself such that its poles lock with corresponding stator poles. With the winding 'A' excited as before, winding 'b' is now to  $F_a$ . the resulting magnetic field F makes an angle of  $45^\circ$ . the rotor consequently moves through  $45^\circ$  in anti clockwise direction, again to cause locking of rotor poles with corresponding stator poles.

While winding 'B' has voltage +V applied to it, winding 'A' is switched off. The rotor then moves through a further  $45^\circ$  in anti-clockwise direction to align itself with stator field  $F_b$ . with voltage +V on winding B, a voltage -V is applied to

winding A. then the stator magnetic field has two components  $F_a$ ,  $F_b$  and their resultant  $F$  makes an angle of  $135^\circ$  position.

In this way it can be seen that, as the pattern of excitation of the state of winding is changed, the rotor moves successively through  $45^\circ$  steps. And completes one full revolution in anti clock-wise direction. A practical PM stepper motor will have  $1.8^\circ$  step angle and 50 tooth on its rotor; there are 8 main poles on the stator, each having five tooth in the pole face. The step angle is given by

$$A = 360 / (N * K) \text{ degrees}$$

Where  $N$  = number of rotor tooth.

$K$  = execution sequence factor.

PM stepper motors have three modes of excitation i.e.,

- Single phase mode
- Two phase mode
- Hybrid mode

**Single phase mode:** in this mode only one of the motor winding is excited at a time. There are four steps in the sequence, the excitation sequence factor  $K=2$ , so that step angle is  $90^\circ$ .

**Two phase mode:** Here both stator phases are excited at a time. There are four steps in the excitation sequence,  $K = 2$  and the step angle is  $90^\circ$ . However, the rotor positions in the two phase mode are  $45^\circ$  away from those in single phase mode.

**Hybrid mode:** this is a combination of single and two phase modes. There are 8 steps in excitation sequence  $= 2$  and step angle  $= 45^\circ$ . a voltage  $+V$  is applied to a stator winding during some steps, which voltage  $V$  is applied during certain other steps. This requires a bipolar regulated power supply capable of yielding  $+V, -V$  and zero outputs and a pair of SPDT switches, which is quite cumbersome. Consequently each of the two stator windings is split into two sections  $A1-A2$ ,  $B1-B2$ . these sections are wound differentially. These winding sections can now be excited from a univocal regulated power supply through switcher  $S1$  to  $S4$ . this type of construction is called bipolar winding construction. Bipolar winding results in reduced winding inductance and consequently improved torque stepping rate.

**Description:** the stepper motor interface uses four transistor pairs (SL 100 and 2N 3055) in a Darlington pair configuration. Each Darlington pair is used to excite the particular winding of the motor connected to 4 pin connector on the interface. The inputs to these transistors are from the 8255 PPI I/O lines of the microprocessor kit or from digital I/O card plugged in the PC. "port A" lower nibble  $PA_0$ ,  $PA_1$ ,  $PA_2$ ,  $PA_3$  are the four lines brought out to the 26 pin FRC male connector ( $J_1$ ) on the interface module. The freewheeling diodes across each winding protect transistors from switching transients.

Theory:

A motor used for moving things in small increments is known as stepper motor.

Stepper motor rotate from one fixed position to next position rather than continuous rotation as in case of other ac or dc motor stepper motors are used in printers to advance the paper from one position to advance the paper from one position to another in steps. They are also used to position the read/write head on the desired track of a floppy disk. To rotate the shaft the stepper motor a sequence of pulses are applied to the windings in a predefined sequence. The number of pulses required for one complete rotation per pulse is given by  $360^\circ/N_T$ , where " $N_T$ " is the number of teeth on rotor. Generally the stepper motor is available with 10 to  $30^\circ$  rotation. They are available with two phases and four phase common field connections.

Instead of rotating smoothly around and around as most motors, stepper motors rotate or step one fixed position to next. Common step size range from  $0.9^\circ$  to  $30^\circ$ . it is stepped from one position to next by changing the currents through the fields in the motor.

The two common field connections are referred to as two phase and four phase. The drive circuitry is simpler in 4 phase stepper. The figure shows a circuitry that can interface a small 4 stepper motor to four microcomputer port lines.

The 7406 buffers are inverting, so. A high on an output port pin turns on current to a winding. The purpose of clamp diodes across each winding is to save transistors from inductive kick. Resistors  $R_1$  and  $R_2$  are current limiting resistors.

Typical parameters of stepper motor:

1. Operating voltage - 12 volts
2. Current rating - 1.2 Amp
3. Step angle -  $1.8^\circ$
4. Step for revolution - 200(No. of teeth on rotor)
5. Torque - 3 kg/cm

### **Working of stepper motor:**

Suppose that  $SW_1$  and  $SW_2$  are turned ON. Turning OFF  $SW_2$  and turning ON  $SW_4$  cause the motor to rotate one step of  $1.8^\circ$  clockwise. Changing to  $SW_4$  and  $SW_3$  ON will cause the motor to rotate  $1.8^\circ$  clockwise another. Changing  $SW_3$  and  $SW_2$  ON will cause another step. To step the motor in counter clock wise direction simply work through the switch sequence in the reverse direction.

The switch pattern for changing from one step to another step in clockwise direction is simply rotated right one position. For counter clockwise direction rotated left one position



**ALGORITHM:**

**Step 1:** Start

**Step 2:** move the control word address 0FFE6 to register DX

**Step 3:** move 80 to AL register.

**Step 4:** locate the contents in AL register to DX register using port out.

**Step 5:** Initialize BX with 07d0

**Step 6:** move port A address ie.,0FFE0 to DX register.

**Step 7:** move 11 to AL register.

**Step 8:** locate the contents in AL register to DX register using port out.

**Step 9:** move 300 to CX register.

**Step 10:** repeat step 8 until the content in CX register becomes equal to zero.

**Step 11:** Rotate carry left through bit.

**Step 12:** Decrement BX by one

**Step 13:** repeat steps from 8 until the content in BX register becomes equal to zero.

**Step 14:** Initialize BX with 07d0

**Step 15:** locate the contents in AL register to DX register using port out.

**Step 16:** move 300 to CX register.

**Step 17:** repeat step 15 until the content in CX register becomes equal to zero.

**Step 18:** Rotate carry left through bit.

**Step 19:** Decrement BX by one

**Step 20:** repeat steps from 15 until the content in BX register becomes equal to zero.

**Step 21:** jump to location / step 8.

**CODE TABLE:**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |



**PROGRAM:**

```
        MOV     DX, 0FFE6
        MOV     AL, 80
        OUT     DX, AL
RPT:    MOV     BX, 07D0
        MOV     DX, 0FFE0
        MOV     AL, 11
BACK:   OUT     DX, AL
        MOV     CX, 0300
L1:     LOOP    L1
        ROL     AL, 1
        DEC     BX
        JNZ     BACK
        MOV     BX, 07D0
        MOV     AL, 11
BACK1:  OUT     DX, AL
        MOV     CX, 0300
L2:     LOOP    L2
        ROR     AL, 1
        DEC     BX
        JNZ     BACK1
        JMP     RPT
```

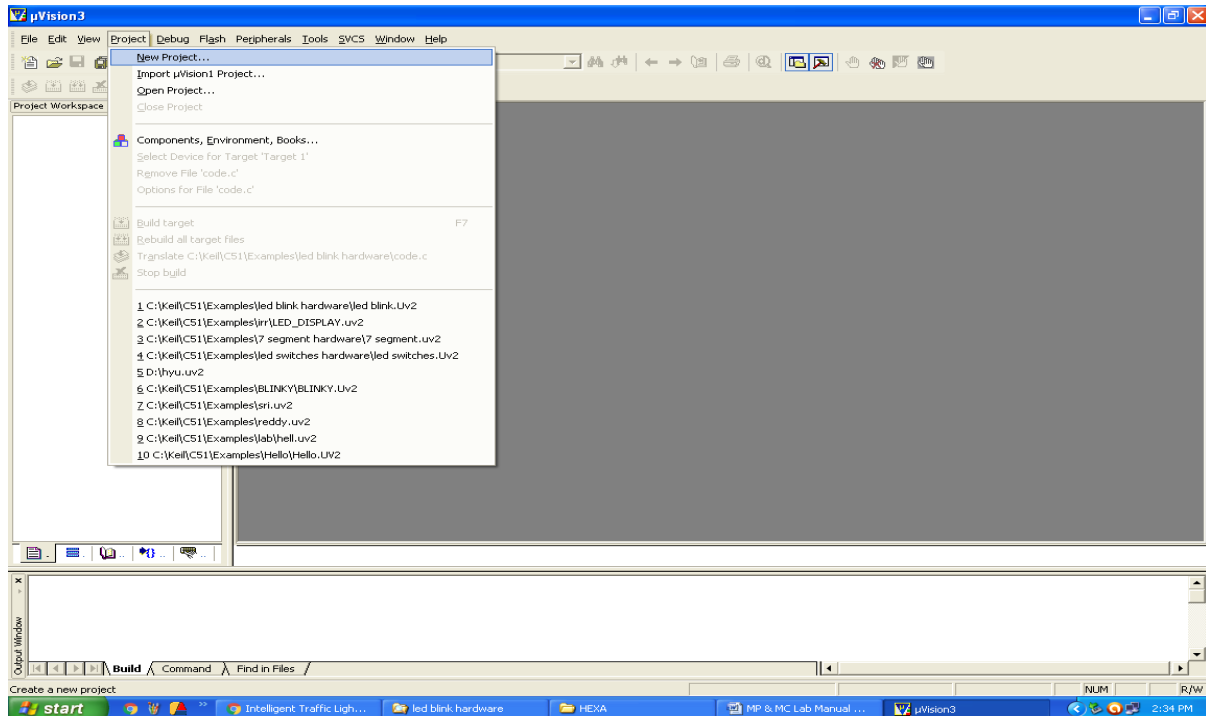


**RESULT:**

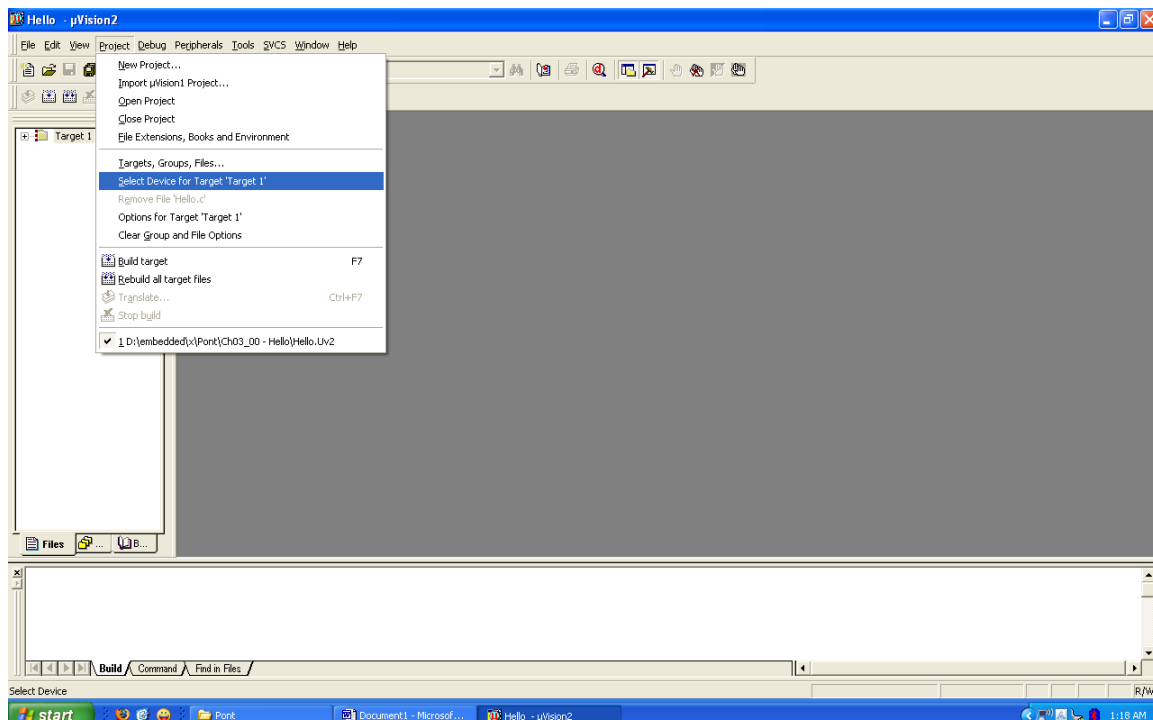


## INTRODUCTION TO KEIL SIMULATION

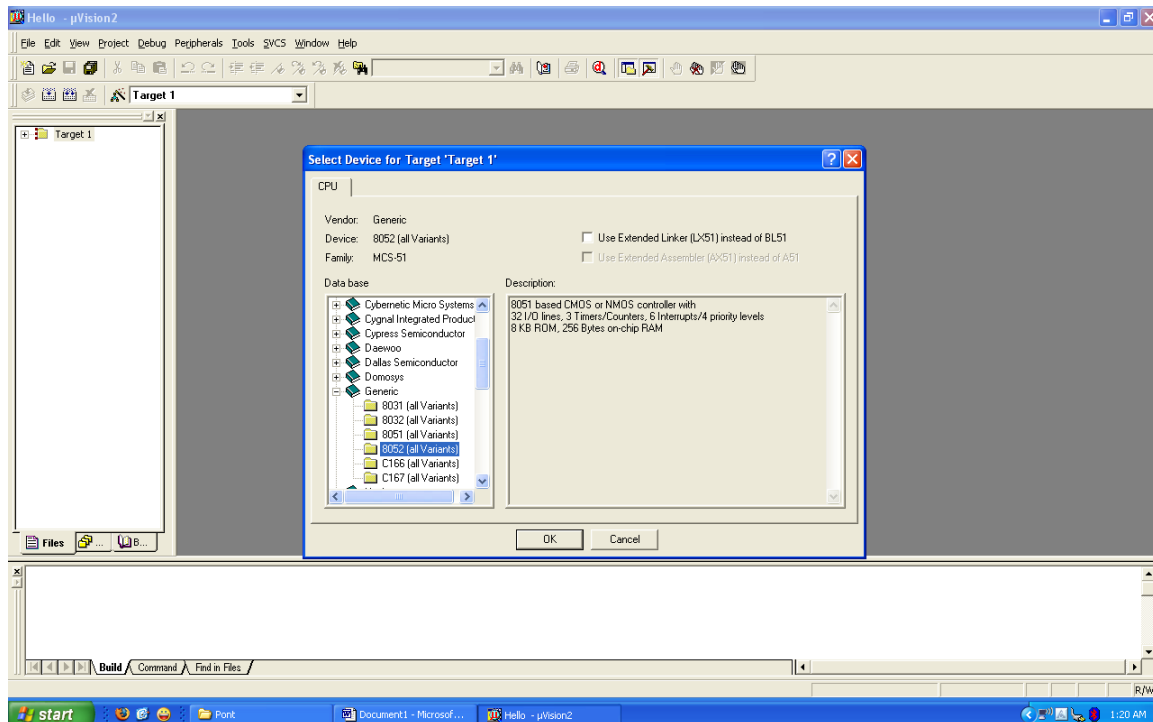
1. Open the Keil  $\mu$ Vision3 on desktop.
2. Go to *file* & open *new project and new file*, type the program & save as file name.c. Go to *Project* – Open Project and browse for saved file and open it.



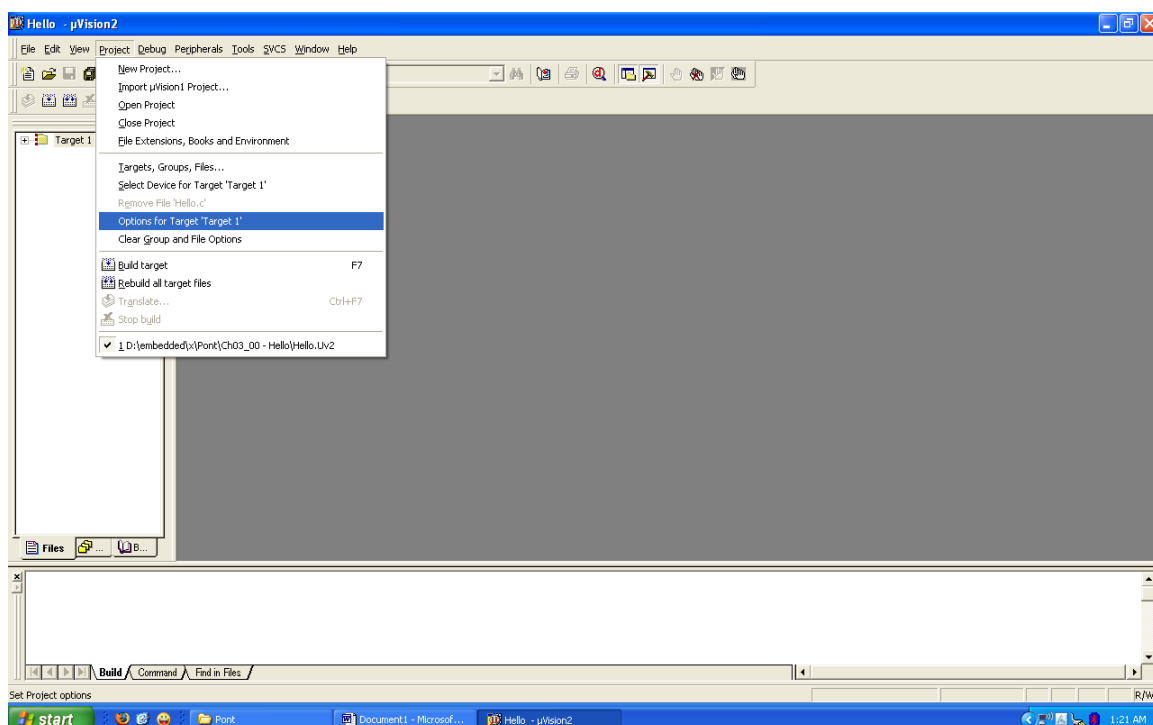
3. Go to *Project* – *Select Device for Target 'Target1'*.



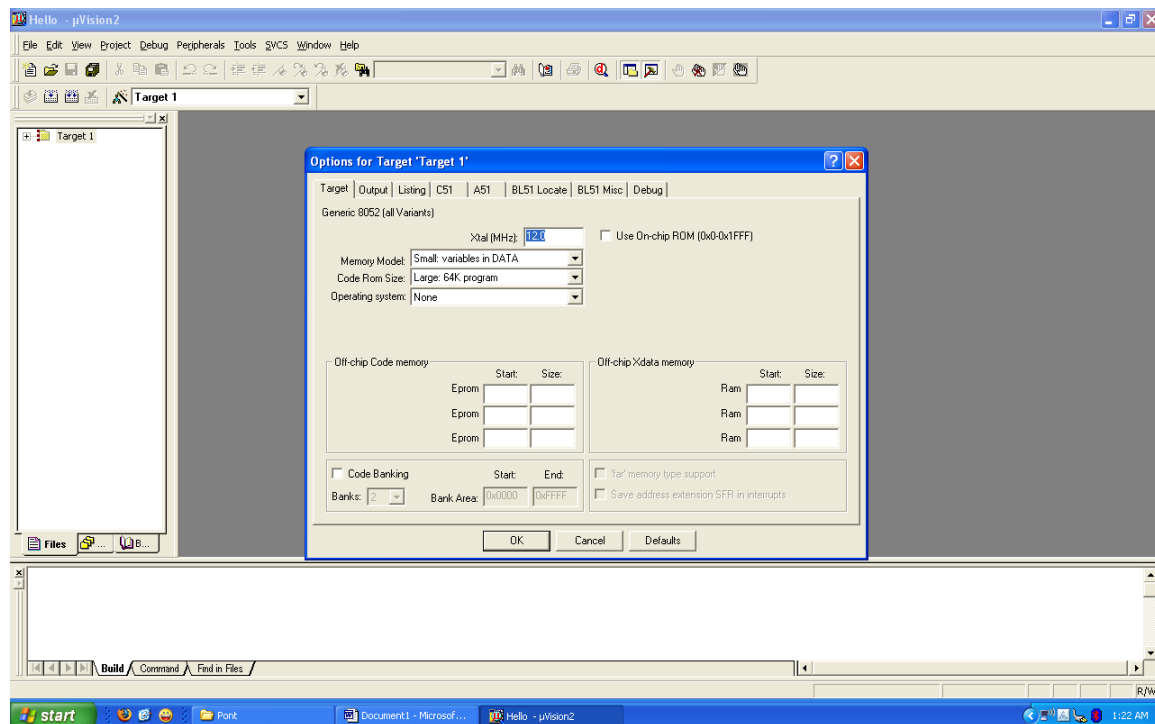
4. Select ATMEL family microcontroller AT89S52 and click OK.



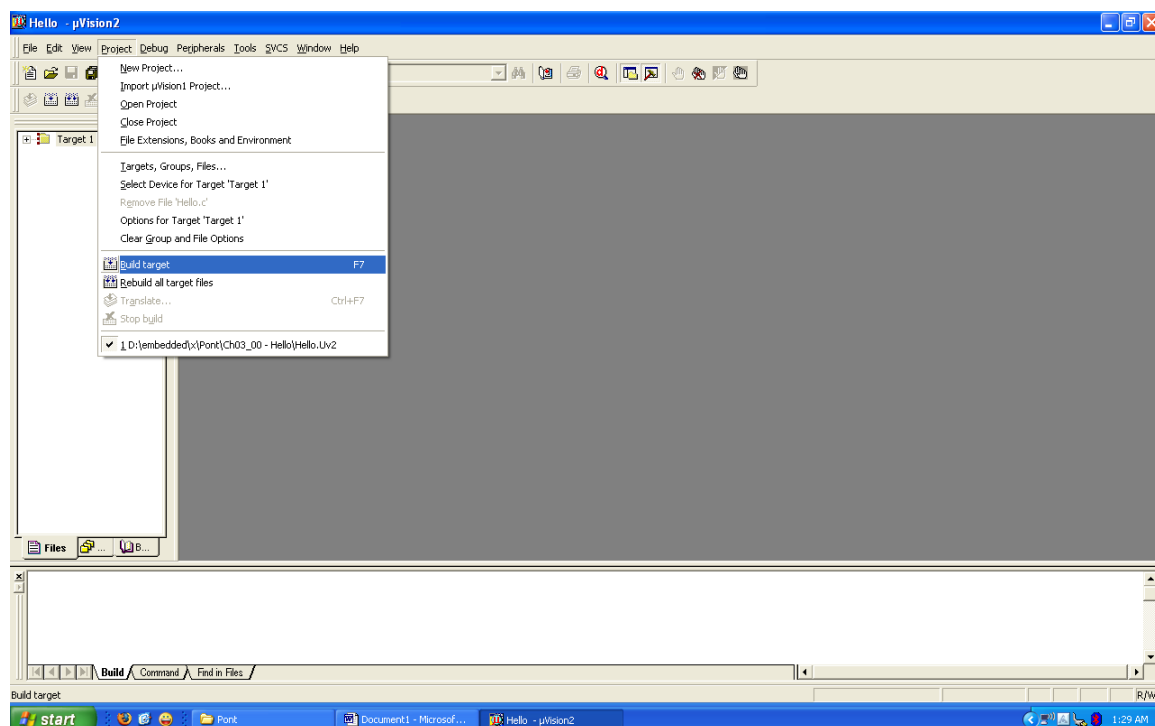
5. Go to project – Options for Target 'Target1'.



6. Make sure that the oscillator frequency is 11.059MHz. Put tick mark on “*use on chip ROM*”. Go to output, put tick mark on “*create HEX file*”. Click *ok*.



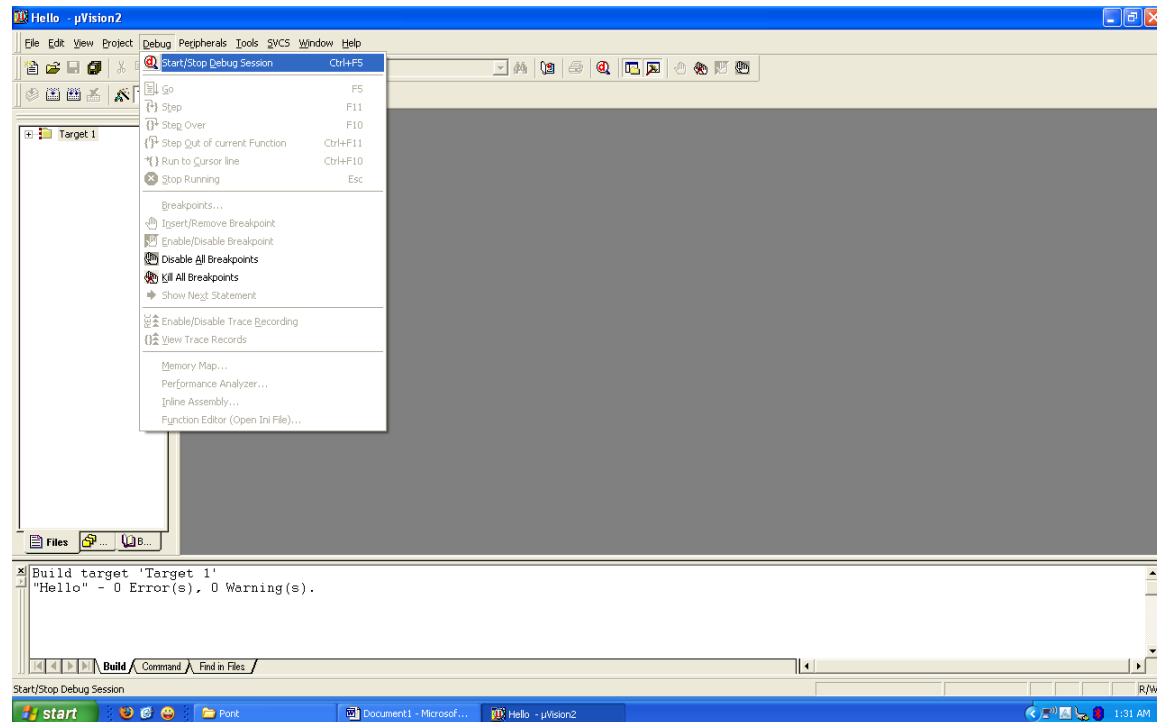
7. Build the target as illustrated in the figure below, check whether HEX file is created or not.



## 8. Running the Simulation

Having successfully built the target, we are now ready to start the debug session and run the simulator.

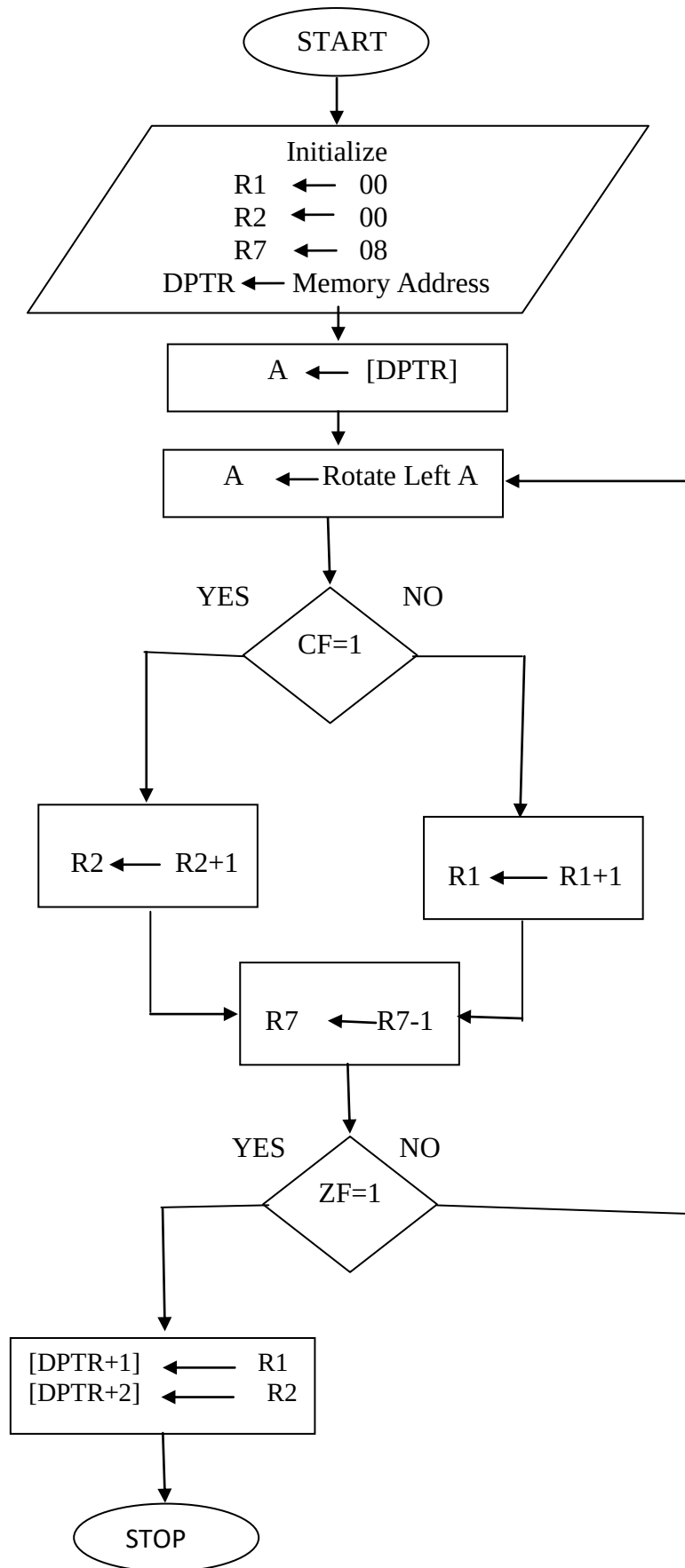
First start a debug session.





## **NUMBER OF 1'S AND NUMBER OF 0'S IN A GIVEN 8-BIT NUMBER**

## FLOW CHART:



**Exp No: 6****Date:****NUMBER OF 1'S AND NUMBER OF 0'S IN A GIVEN 8-BIT NUMBER****AIM:** Assembly Language Program to Find Number of 1's and 0's**PORTS USED:** None**REGISTERS USED:** R1, R2, R7, A, DPTR**ALGORITHM:****Step 1:** Start**Step 2:** Initialize R1 and R2 registers with zeros**Step 3:** Load the value 08H into R7 register**Step 4:** Initializing base address location where given number is stored**Step 5:** Copying value from memory location at which DPTR is pointing**Step 6:** Rotate Accumulator Left Through Carry**Step 7:** if CY=1 then Jump to Step 10**Step 8:** Increment the content of R1 register**Step 9:** Jump to Step 11**Step 10:** Increment the content of R2 register**Step 11:** Decrement register R7 and jump if not Zero to step 6**Step 12:** Moving the content of R1 to register A**Step 13:** Increment the content of DPTR register**Step 14:** Moving the content of register A to external RAM**Step 15:** Moving the content of R2 to register A**Step 16:** Increment the content of DPTR register**Step 17:** Moving the content of register A to external RAM**Step 18:** Repeat step 18

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**PROGRAM:**

```
        MOV R1, #00
        MOV R2, #00
        MOV R7, #08
MOV DPTR, #4500H
MOVX A, @DPTR
AGAIN:  RLC A
        JC NEXT
        INC R1
        SJMP HERE
NEXT:   INC R2
HERE:   DJNZ R7, AGAIN
        MOV A, R1
        INC DPTR
        MOVX @DPTR, A
        MOV A, R2
        INC DPTR
        MOVX @DPTR, A
HLT:    SJMP HLT
        END
```



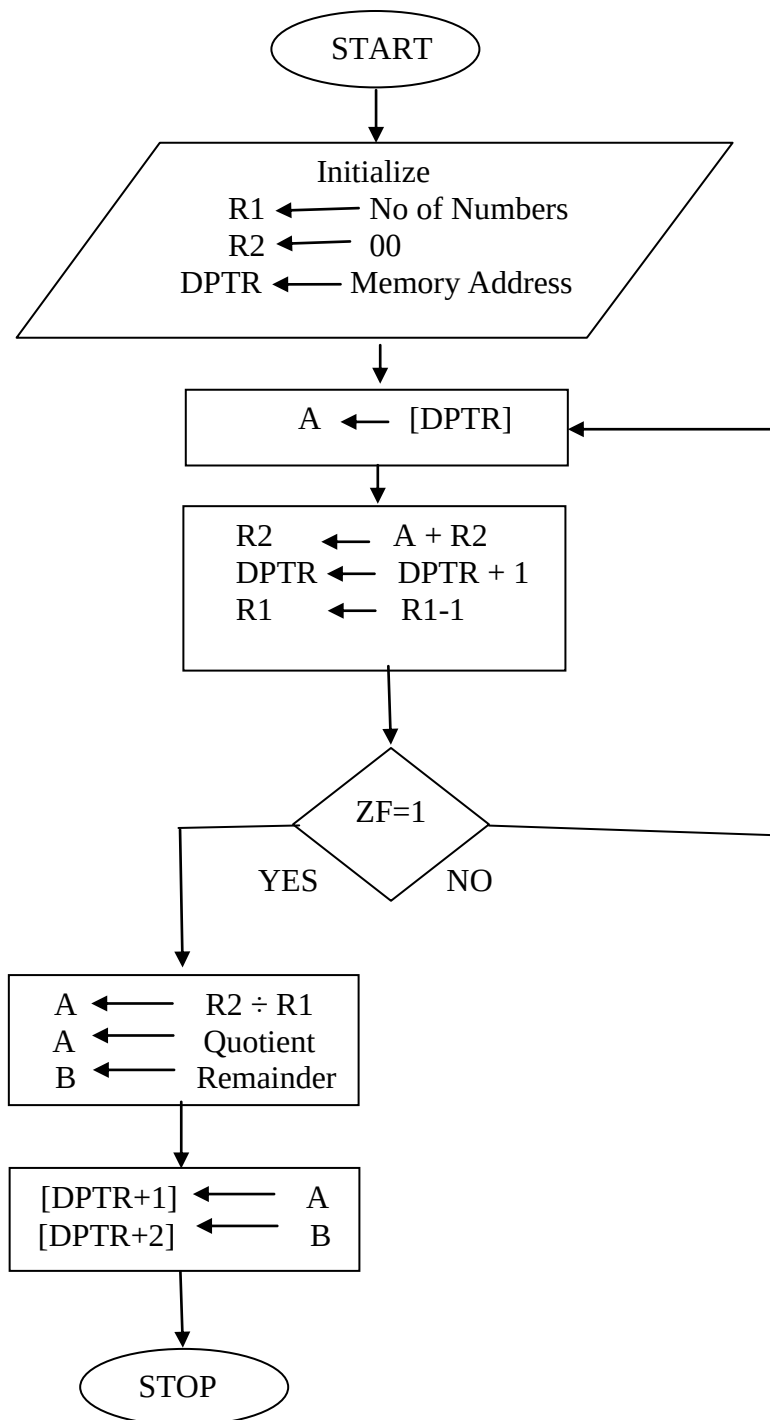
**RESULT:**





# **AVERAGE OF N-NUMBERS**

## FLOW CHART:



**Exp No: 7****Date:**

### **AVERAGE OF N-NUMBERS**

**AIM:** Assembly language program to find the average of N numbers.

**PORTS USED:** None

**REGISTERS USED:** A, B, R1, R2, DPTR

**ALGORITHM:**

**Step 1:** Start

**Step 2:** Load the value 05H into R1 register

**Step 3:** Moving the content of R1 to register B

**Step 4:** Initializing R2 register to zero

**Step 5:** Initializing base address location where given number is stored

**Step 6:** Copying value from memory location at which DPTR is pointing

**Step 6:** Rotate Accumulator Left Through Carry

**Step 7:** Adds the content of Accumulator and content of R2 register.

**Step 8:** Moving the content of A to register R2.

**Step 9:** Increment the content of DPTR register

**Step 10:** Decrement register R1 and jump if not Zero to step 6

**Step 11:** Moving the content of register R2 to register A.

**Step 12:** Divide Accumulator by B

**Step 13:** Moving the content of register A to external RAM

**Step 14:** Increment the content of DPTR register

**Step 15:** Moving the content of B to register A

**Step 16:** Moving the content of register A to external RAM

**Step 17:** Repeat step17

**CODE TABLE:**

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**PROGRAM:**

MOV R1, #05

MOV B, R1

MOV R2, #00H

MOV DPTR, #5500H

UP: MOVX A, @DPTR

ADD A, R2

MOV R2, A

INC DPTR

DJNZ R1, UP

MOV A, R2

DIV AB

MOVX @DPTR, A

INC DPTR

MOV A, B

MOVX @DPTR, A

HLT: SJMP HLT

END

**MANUAL CALCULATIONS:**

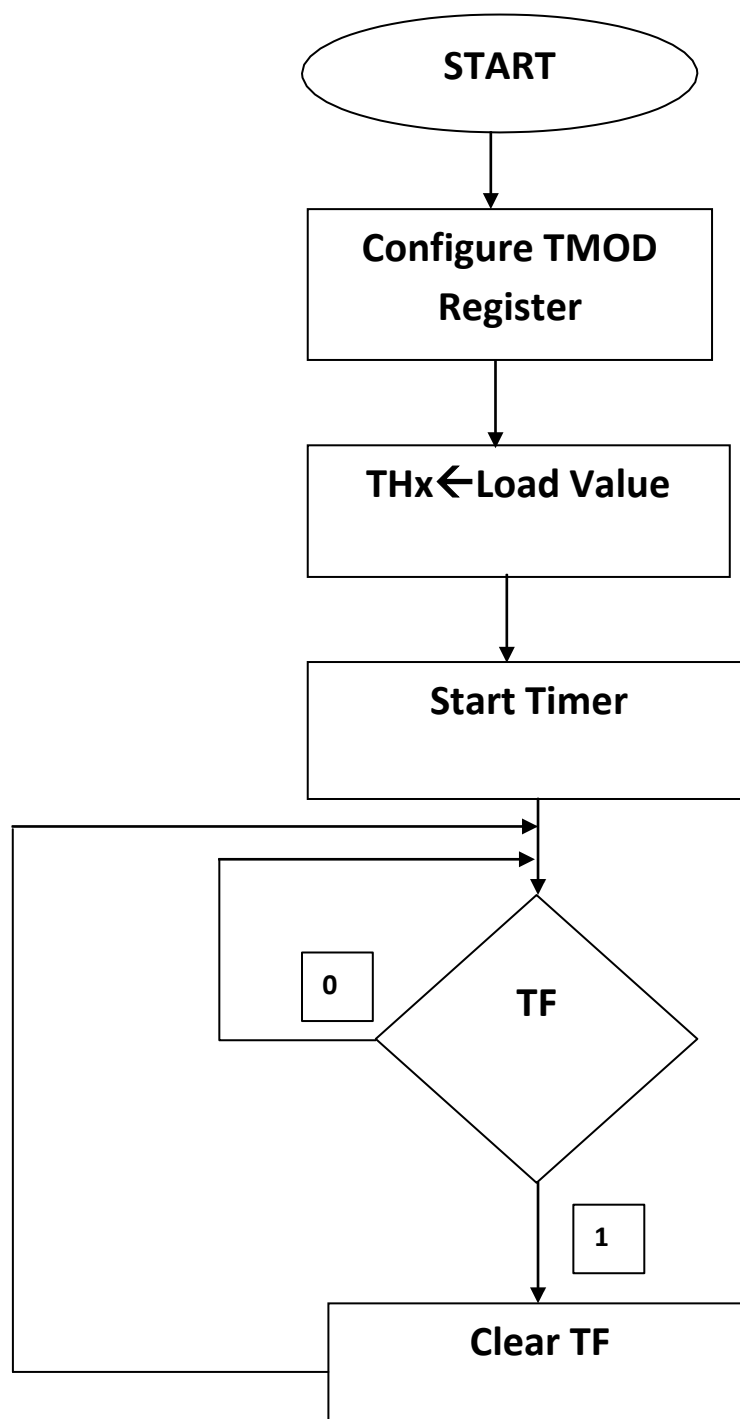
**RESULT:**





## **TIMER CONTROL BASED SQUARE WAVE GENERATION PROGRAM USING 8051**

## FLOW CHART:



Exp No: 8

Date:

## TIMER CONTROL BASED SQUARE WAVE GENERATION PROGRAM USING 8051.

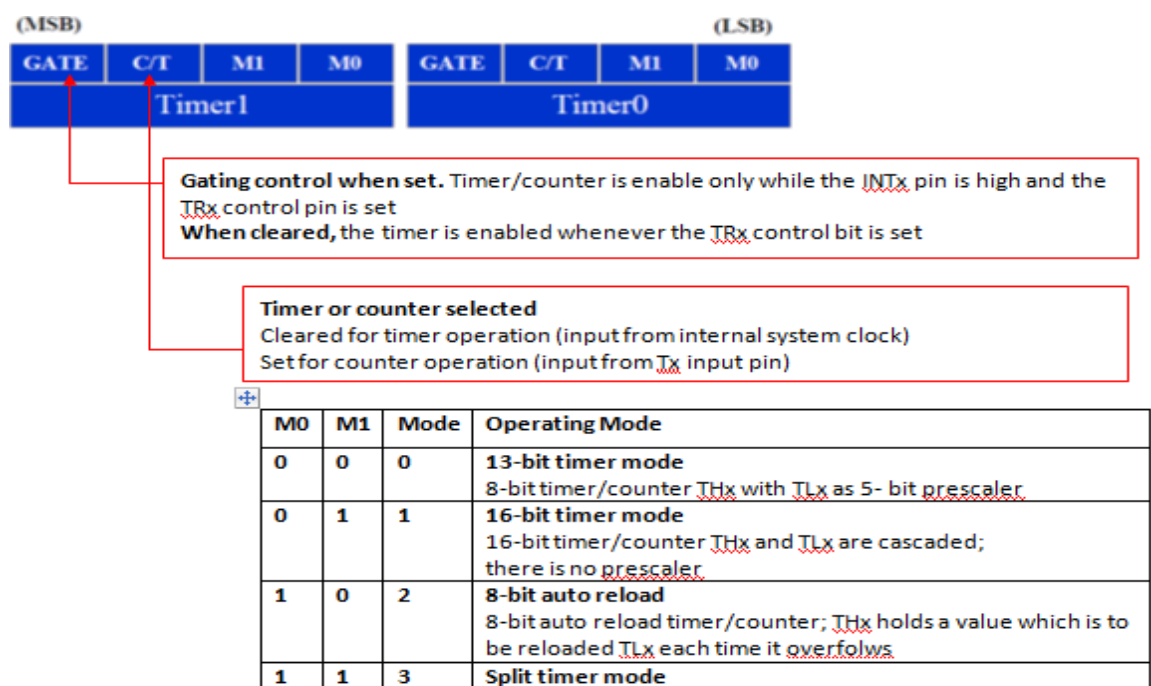
**AIM:** Write an Assembly language program to create a square wave of 50% duty cycle on P1.5

**TOOLS USED:** Keil Software

**PORTS USED:** P1

**REGISTERS USED:** TMOD, TLX, THX

TMOD is an 8-bit register



### ALGORITHM:

1. Configure TMOD register( Select Timer and Mode of operation)
2. Load registers THX, TLX with initial count.
3. Start the Timer
4. Monitor TF for high
5. Stop the Timer
6. Clear TF
7. Repeat Step '2'

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**PROGRAM:**

MOV TMOD, #01 ;Timer 0, mode 1(16-bit mode)

HERE: MOV TL0, #0F2H ;TL0=F2H, the low byte

MOV TH0, #0FFH ;TH0=FFH, the high byte

CPL P1.5 ;toggle P1.5

ACALL DELAY

SJMP HERE

DELAY: SETB TR0 ;start the timer 0

AGAIN :JNB TF0, AGAIN ;monitor timer flag 0 -until it rolls over

CLR TR0 ;stop timer 0

CLR TF0 ;clear timer 0 flag

RET



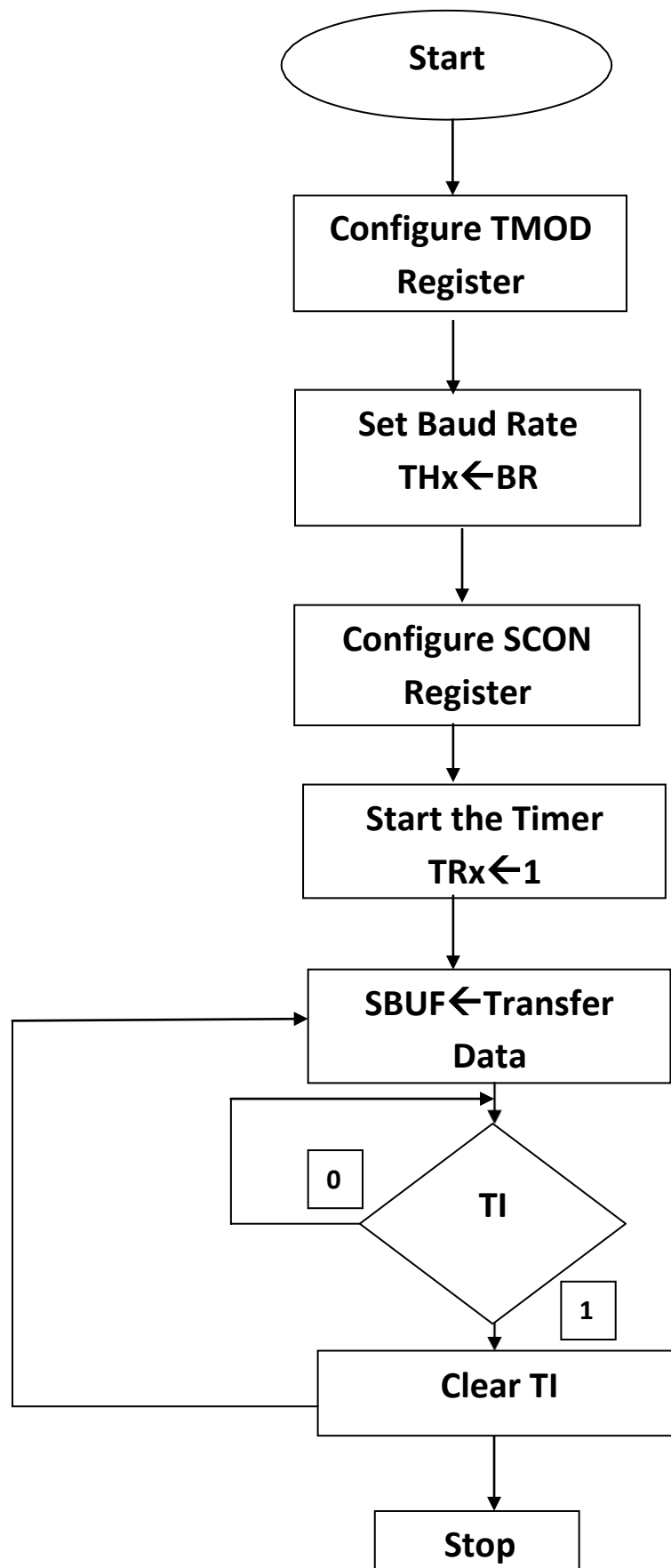
**RESULT:**





# **INTERFACE OF UART WITH 8051**

## FLOW CHART:



Exp No: 9

Date:

## INTERFACE OF UART WITH 8051

**AIM:** Write an Assembly language program for the 8051 to transfer letter 'A' serially at 9600 baud rate, continuously

**TOOLS USED:** Keil Software

**PORTS USED:** None

**REGISTERS USED:** TMOD, THx,

SCON, SBUF, TMOD:



**Gating control when set.** Timer/counter is enable only while the INTx pin is high and the TRx control pin is set  
**When cleared,** the timer is enabled whenever the TRx control bit is set

### Timer or counter selected

Cleared for timer operation (input from internal system clock)  
 Set for counter operation (input from TRx input pin)



| M0 | M1 | Mode | Operating Mode                                                                                                                                         |
|----|----|------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0  | 0  | 0    | <b>13-bit timer mode</b><br>8-bit timer/counter <u>THx</u> with <u>TLx</u> as 5-bit <u>prescaler</u>                                                   |
| 0  | 1  | 1    | <b>16-bit timer mode</b><br>16-bit timer/counter <u>THx</u> and <u>TLx</u> are cascaded;<br>there is no <u>prescaler</u>                               |
| 1  | 0  | 2    | <b>8-bit auto reload</b><br>8-bit auto reload timer/counter; <u>THx</u> holds a value which is to be reloaded <u>TLx</u> each time it <u>overflows</u> |
| 1  | 1  | 3    | <b>Split timer mode</b>                                                                                                                                |

**SCON:**

| SM0 | SM1 | SM2 | REN | TB8 | RB8 | TI | RI |
|-----|-----|-----|-----|-----|-----|----|----|
|-----|-----|-----|-----|-----|-----|----|----|

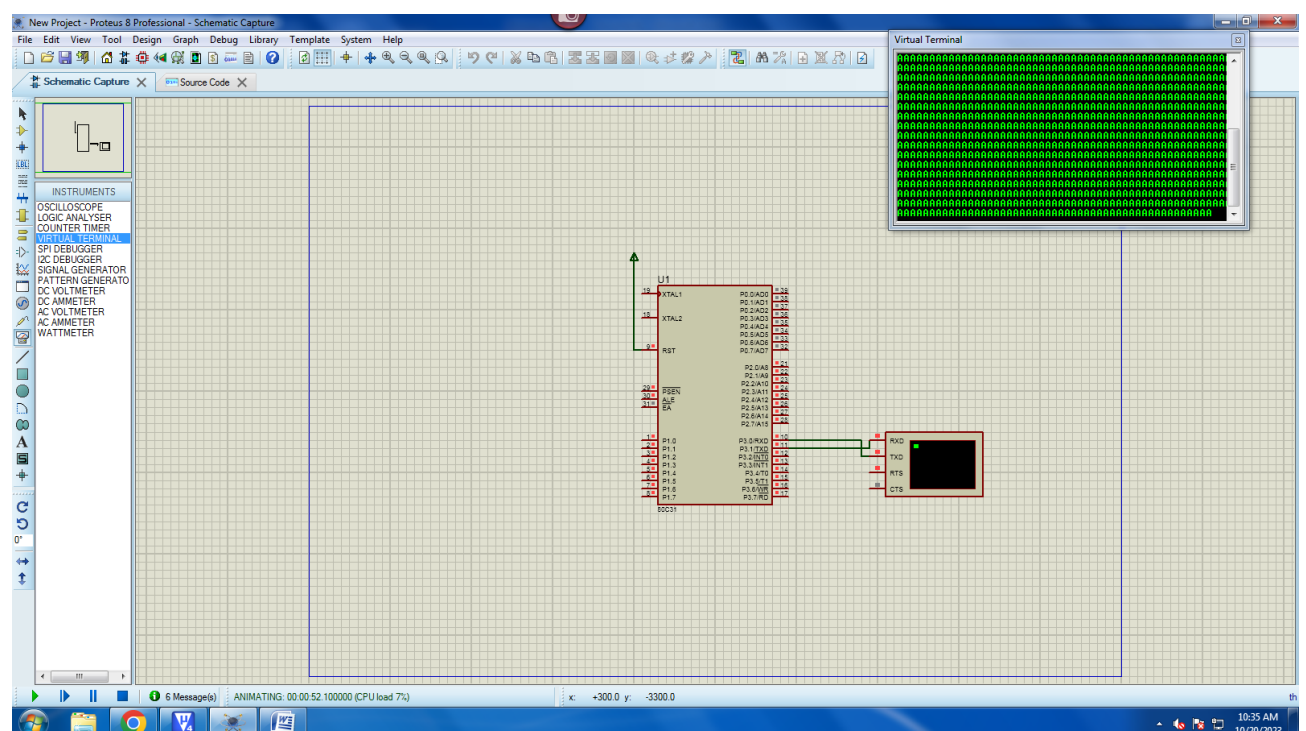
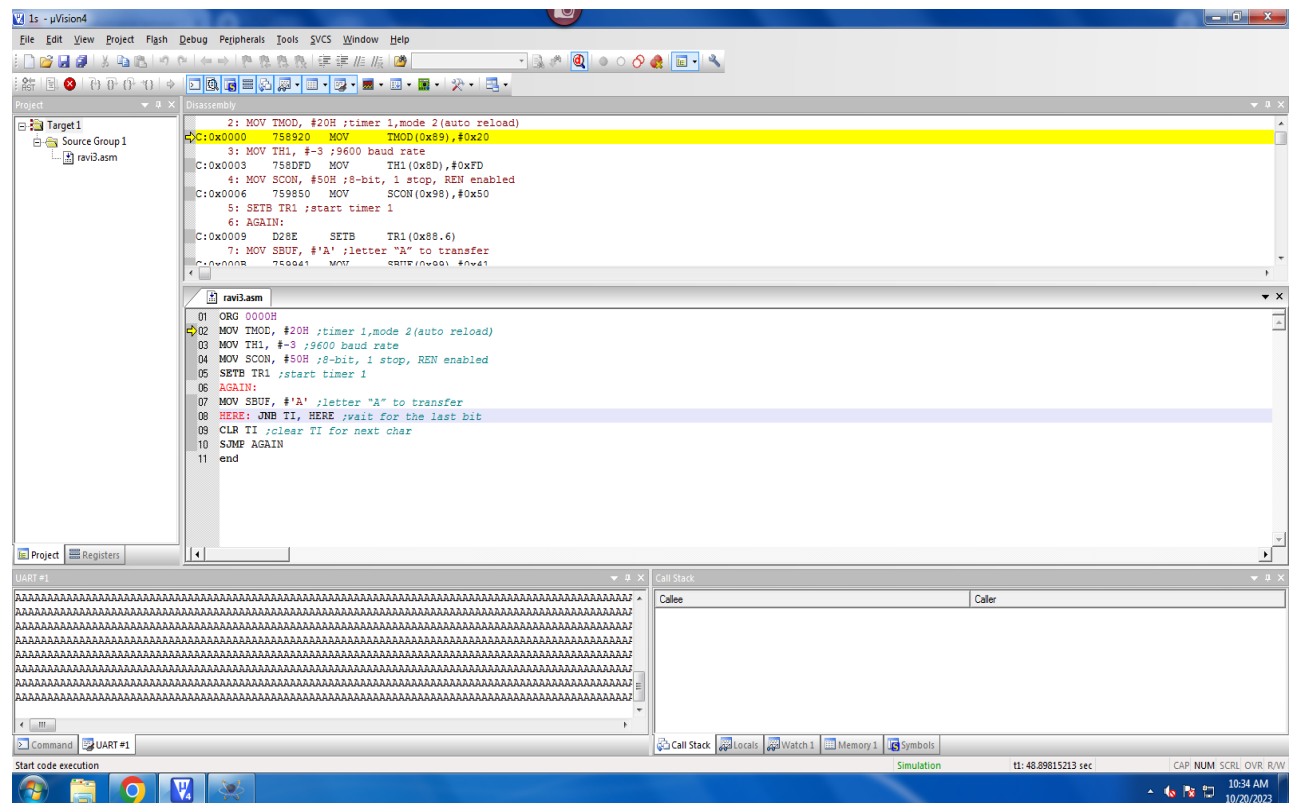
|            |        |                                                                                           |
|------------|--------|-------------------------------------------------------------------------------------------|
| <b>SM0</b> | SCON.7 | Serial port mode specifier                                                                |
| <b>SM1</b> | SCON.6 | Serial port mode specifier                                                                |
| <b>SM2</b> | SCON.5 | Used for multiprocessor communication                                                     |
| <b>REN</b> | SCON.4 | Set/cleared by software to enable/disable reception                                       |
| <b>TB8</b> | SCON.3 | Not widely used                                                                           |
| <b>RB8</b> | SCON.2 | Not widely used                                                                           |
| <b>TI</b>  | SCON.1 | Transmit interrupt flag. Set by HW at the begin of the stop bit mode 1. And cleared by SW |
| <b>RI</b>  | SCON.0 | Receive interrupt flag. Set by HW at the begin of the stop bit mode 1. And cleared by SW  |

**ALGORITHM:**

1. Configure TMOD Register ( Select Timer and Mode of Operation)
2. Load THx with value to set baud rate
3. Configure SCON register according to framing of data
4. Start the timer
5. Load the transfer data in to SBUF
6. Monitor the TI bit till last bit transmitted
7. Clear the TI for next character
8. Repeat step '5'

**PROGRAM:**

```
ORG 0000H
MOV TMOD, #20H           ; timer 1,mode 2(auto reload)
MOV TH1, #-3             ; 9600 baud rate
MOV SCON, #50H           ;8-bit, 1 stop, REN enabled
SETB TR1                 ;start timer 1
AGAIN: MOV SBUF, #'A'    ; letter "A" to transfer
HERE: JNB TI, HERE       ;wait for the last bit
CLR TI                   ;clear TI for next char
SJMP AGAIN
END
```



**RESULT:**





# **INTERFACE LCD TO 8051**



**Exp No: 10** **Date:**  
**INTERFACE LCD TO 8051 USING KEIL & PROTEUS**  
**SOFTWARE**

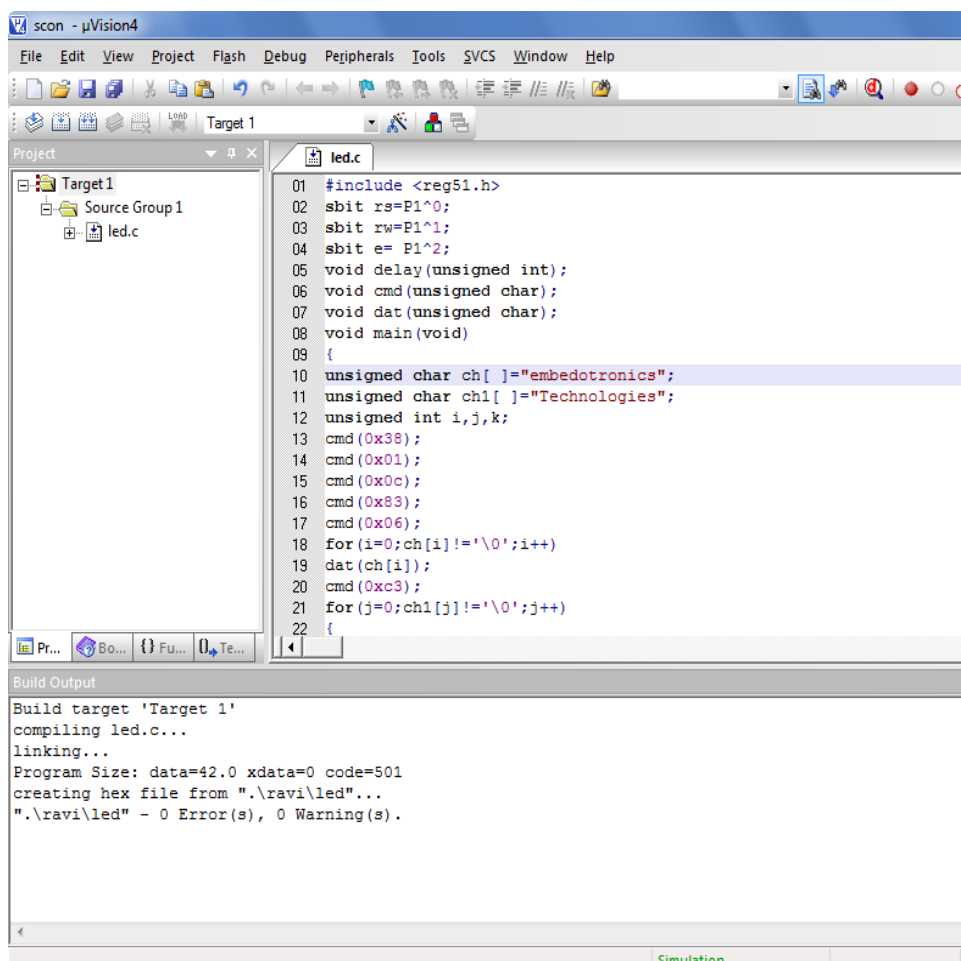
**AIM:** Write an Assembly language program to interface LCD to 8051.

**TOOLS USED:** Keil Software

**PORTS USED:** None

**PROGRAM:**

```
#include <reg51.h>
sbit rs=P1^0;
sbit rw=P1^1;
sbit e= P1^2;
void delay(unsigned int);
void cmd(unsigned char);
void dat(unsigned char);
void main(void)
{
    unsigned char ch[ ]="ADITYA";
    unsigned char ch1[ ]="ECE";
    unsigned int i,j,k;
    cmd(0x38);
    cmd(0x01);
    cmd(0x0c);
    cmd(0x83);
    cmd(0x06);
    for(i=0;ch[i]!='\0';i++)
        dat(ch[i]);
    cmd(0xc3);
    for(j=0;ch1[j]!='\0';j++)
    {
        dat(ch1[j]);
    }
    while(1){
        for(k=0;k<16;k++)
        {
            cmd(0x1c);
        }
    }
    void
    delay(unsigned
    int t)
    {
        unsigned int i,j;
        e=1;
        for(i=0;i<t;i++)
            for(j=0;j<1275;j+
            +);
        e=0;
    }
}
```



The screenshot displays the sicon - µVision4 IDE interface. The main window shows the source code for a file named `led.c`. The code includes headers, defines pins, and implements a main function that sends commands and data to an LED. The Build Output window at the bottom shows the successful compilation and linking of the target 'Target 1'.

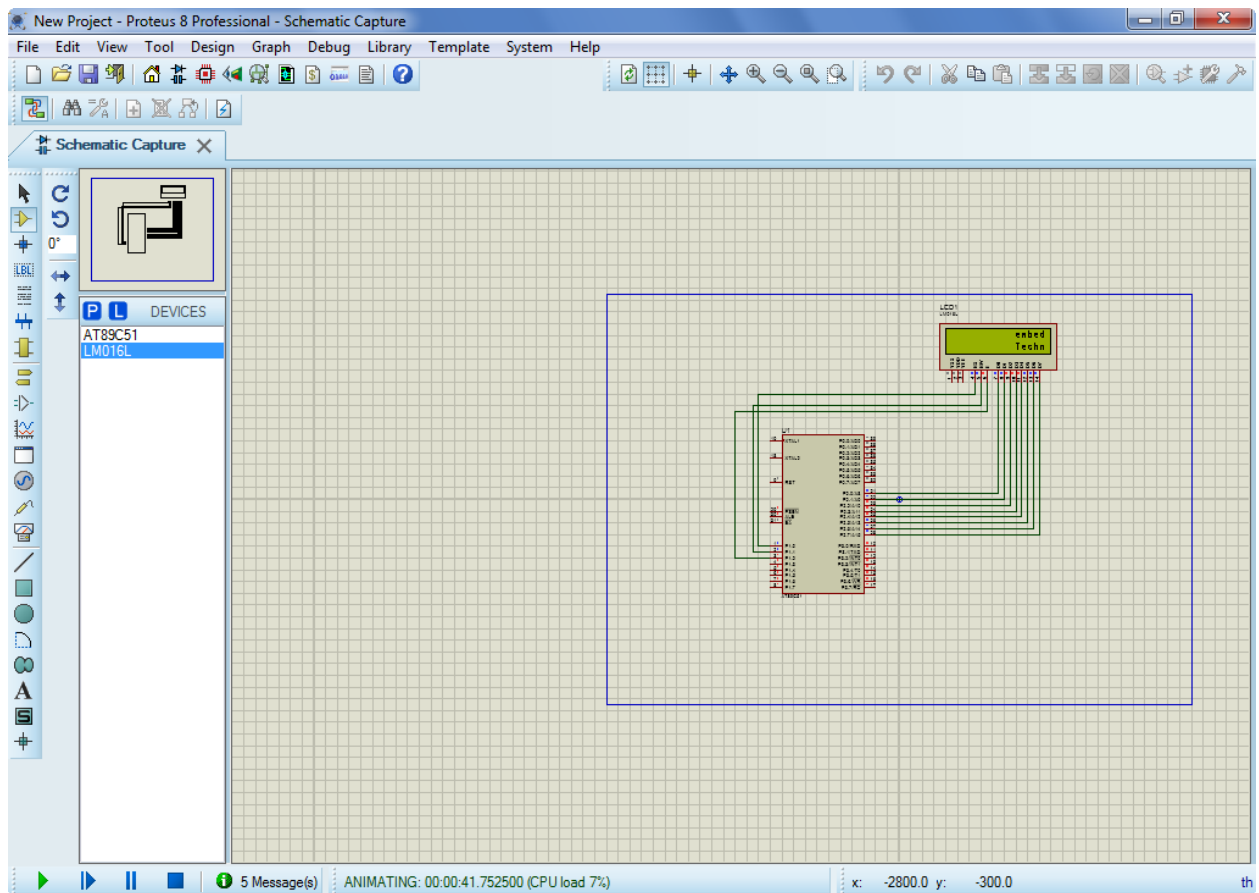
```
01 #include <reg51.h>
02 sbit rs=P1^0;
03 sbit rw=P1^1;
04 sbit e= P1^2;
05 void delay(unsigned int);
06 void cmd(unsigned char);
07 void dat(unsigned char);
08 void main(void)
09 {
10     unsigned char ch[ ]="embedotronics";
11     unsigned char chl[ ]="Technologies";
12     unsigned int i,j,k;
13     cmd(0x38);
14     cmd(0x01);
15     cmd(0x0c);
16     cmd(0x83);
17     cmd(0x06);
18     for(i=0;ch[i]!='\0';i++)
19         dat(ch[i]);
20     cmd(0xc3);
21     for(j=0;chl[j]!='\0';j++)
22     {
```

Build Output

```
Build target 'Target 1'
compiling led.c...
linking...
Program Size: data=42.0 xdata=0 code=501
creating hex file from ".\ravi\led"...
".\ravi\led" - 0 Error(s), 0 Warning(s).
```

```
}  
  
void  
cmd(unsigned  
char ch)  
{  
    rs=0;  
    rw=0;  
    P2=ch;  
    delay(20);  
}
```

```
void  
dat(unsigned  
char ch)  
{  
    rs=1;  
    rw=0;  
    P2=ch;  
    delay(20);  
}
```



**RESULT:**





# **AUGMENTED EXPERIMENTS**



# **INTERFACE OF 8086 WITH TRAFFIC LIGHT CONTROLLER**



**Exp No: 11****Date:**

## **INTERFACE OF 8086 WITH TRAFFIC LIGHT CONTROLLER**

**AIM:** Write an Assembly language program to Interface Traffic Light Controller with an 8086 microprocessor.

**APPARATUS REQUIRED:**

1. 8086 microprocessor kit.
2. Traffic light interfacing cord.

**REGISTERS USED:** General Purpose Registers

**PORTS USED:** Port 0, Port 1, Port 2, Port 3.

**ALGORITHM:**

- 1. Setup and Initialization**  
Initializes and writes to a specific port.
- 2. Loop and Data Output**  
Enters a loop, outputting bytes from a data table to a port with incremental delays.
- 3. Delay Subroutine**  
Executes a delay subroutine to control timing between port outputs.
- 4. Data Table (PORTS)**  
Terminates the loop once the end of the data table is reached.

| Physical Address  |                | Label | Hex Code | Mnemonic operand | Comments |
|-------------------|----------------|-------|----------|------------------|----------|
| Segment : Address | Offset Address |       |          |                  |          |
|                   |                |       |          |                  |          |

**PROGRAM:**

```
START: MOV AL, 80
      MOV DX, 0FFE6
      OUT DX, AL
AGAIN: MOV SI, 2038
NEXTST: MOV AL, [SI]
      MOV DX, 0FFE0
      OUT DX, AL
      INC SI
      ADD DL, 02
      MOV AL, [SI]
      OUT DX, AL
      INC SI
      ADD DL, 02
      MOV AL, [SI]
      OUT DX, AL
      INC SI
      CALL 2029
      CMP SI, 2056
      JNZ 2009
      JMP 2006
DELAY: MOV CX, 03FF
      DLY 5: PUSH CX
      MOV CX, 03FF
      DLY 10: NOP
      LOOP 2030
      POP CX
      LOOP 202C
      RET
PORTS: ORG 2038
      DB 88, 83
      DB 0F2, 88
      DB 87,0F2
      DB 38, 88
      DB 0F4, 78
      DB 88,0F4
      DB 83, 88
      DB 0F8, 87
      DB 88,0F8
      DB 88, 38
      DB 0F1, 88
      DB 78,0F1
      DB 88, 88
      DB 00, 88
      DB 88, 00
```

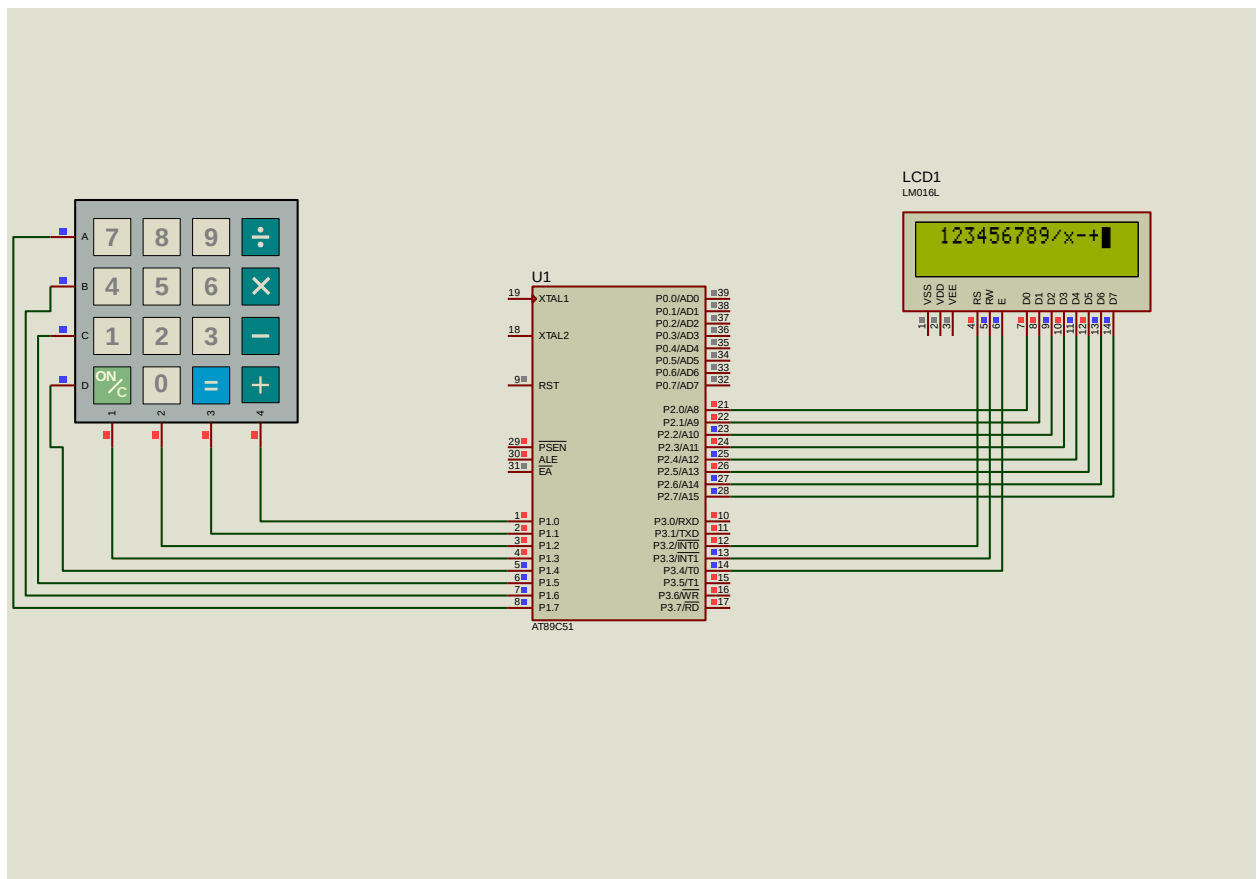




**RESULT:**



# **INTERFACE KEYBOARD TO 8051**

**CIRCUIT DIAGRAM:**

**Exp No:12****Date:**

## INTERFACE KEYBOARD TO 8051

**Aim:** Write a C program to Interface keyboard to 8051 Microcontroller.

**Tools Used:** Keil Software

**Ports Used:** Port 1, Port 2, Port 3.

**Algorithm:**

**Step 1:** Define ports and pins for LCD and keypad.

**Step 2:** Initialize the LCD.

**Step 3:** Start scanning columns in sequence.

**Step 4:** For each column, check each row for a button press.

**Step 5:** Identify the pressed key and display it on the LCD.

**Step 6:** Add a delay to stabilize the keypad inputs.

**Step 7:** Repeat the scan and display process.

**PROGRAM:**

```
#include<reg51.h>

#define display_port P2    //Data pins connected to port 2 on microcontroller

sbit rs = P3^2; //RS pin connected to pin 2 of port 3
sbit rw = P3^3; // RW pin connected to pin 3 of port 3
sbit e = P3^4; //E pin connected to pin 4 of port 3
sbit C4 = P1^0;    // Connecting keypad to Port 1
sbit C3 = P1^1;
sbit C2 = P1^2;
sbit C1 = P1^3;
sbit R4 = P1^4;
sbit R3 = P1^5;
sbit R2 = P1^6;
sbit R1 = P1^7;

void msdelay(unsigned int time) // Function for creating delay in milliseconds.
{
    unsigned i,j ;
```

```
    for(i=0;i<time;i++)
    for(j=0;j<1275;j++);
}

void lcd_cmd(unsigned char command) //Function to send command instruction to LCD
{
    display_port = command;
    rs= 0;
    rw=0;
    e=1;
    msdelay(1);
    e=0;
}

void lcd_data(unsigned char disp_data) //Function to send display data to LCD
{
    display_port = disp_data;

    rs= 1;

    rw=0;

    e=1;

    msdelay(1);

    e=0;
}

void lcd_init() //Function to prepare the LCD and get it ready
{
    lcd_cmd(0x38); // for using 2 lines and 5X7 matrix of LCD

    msdelay(10);

    lcd_cmd(0x0F); // turn display ON, cursor blinking

    msdelay(10);

    lcd_cmd(0x01); //clear screen

    msdelay(10);

    lcd_cmd(0x81); // bring cursor to position 1 of line 1

    msdelay(10);
}
```

```
void row_finder1() //Function for finding the row for column 1
{
    R1=R2=R3=R4=1;
    C1=C2=C3=C4=0;
    if(R1==0)
        lcd_data('7');
    if(R2==0)
        lcd_data('4');
    if(R3==0)
        lcd_data('1');
    if(R4==0)
        lcd_data('e');
}

void row_finder2() //Function for finding the row for column 2
{
    R1=R2=R3=R4=1;
    C1=C2=C3=C4=0;
    if(R1==0)
        lcd_data('8');
    if(R2==0)
        lcd_data('5');
    if(R3==0)
        lcd_data('2');
    if(R4==0)
        lcd_data('0');
}

void row_finder3() //Function for finding the row for column 3
{
```

```
R1=R2=R3=R4=1;

C1=C2=C3=C4=0;

if(R1==0)
    lcd_data('9');
if(R2==0)
    lcd_data('6');
if(R3==0)
    lcd_data('3');
if(R4==0)
    lcd_data('=');
}

void row_finder4() //Function for finding the row for column 4
{

    R1=R2=R3=R4=1;

    C1=C2=C3=C4=0;

    if(R1==0)
        lcd_data('/');
    if(R2==0)
        lcd_data('x');
    if(R3==0)
        lcd_data('-');
    if(R4==0)
        lcd_data('+');
}

void main()
{
    lcd_init();

    while(1)
    {
        msdelay(30);
```



```
C1=C2=C3=C4=1;

R1=R2=R3=R4=0;

if(C1==0)
    row_finder1();
else if(C2==0)
    row_finder2();
else if(C3==0)
    row_finder3();
else if(C4==0)
    row_finder4();
}

}
```

**RESULT:**